飞腾嵌入式裸机 SDK MicroPython 用户使用手册

（V1.0）

飞腾信息技术有限公司
www.phytium.com.cn
2024 年 01 月 23 日

版权所有 © 飞腾信息技术有限公司 2024。 保留一切权利。

未经本公司同意，任何单位、公司或个人不得擅自复制、翻译、摘抄本文档内容的部分或全部，不得以任何方式或途径进行传播和宣传。

商标声明

Phytium 和其他飞腾商标均为飞腾信息技术有限公司的商标。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

特别提示

本文档仅作为使用指导，飞腾对本文档内容不做任何明示或暗示的声明或保证。本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

由于产品版本升级或其他原因，本文档内容会不定期进行更新，如有变更，恕不另行通知。

最新技术资源

飞腾嵌入式软件开源社区： https://gitee.com/phytium_embedded

或者访问飞腾软件开发者平台： https://service.phytium.com.cn/developer/

目录

1. MicroPython 概要 3
2. MicroPython 仓库介绍 3
2.1. 裸机 MicroPython 仓库结构 3
2.2. 裸机 Micropython 特性支持 4
2.3. 例程使用说明及介绍 6
3.基于裸机 + MicroPython 开发指南 8
3.1 如何添加用户函数 8
3.2 如何添加用户模块 10
4.基于裸机 + MicroPython 开发前瞻 11
A 参考文献 12
B 更新记录 12

Phytium 飞腾

1. MicroPython 概要

MicroPython 是 python 的一个精简版本，它是为了运行在单片机等性能有限的微控制器中，最小体积仅 256K，运行时仅需 16K 内存。

MicroPython 可以让有 Python 语言经验但没有 C 语言基础、嵌入式基础和硬件基础的人，更快捷的接触嵌入式，控制自己手里的 MCU。

利用飞腾 Phytium Standalone Sdk 以及第三方库 Micropython，用户可以轻松便捷的使用 Python 语言操控硬件设备。

更多关于 MicroPython 介绍请参考 http://www.micropython.org/

2. MicroPython 仓库介绍

本项目基于开源项目 Micropython v1.22 开发：

GitHub - micropython/micropython: MicroPython - a lean and efficient Python implementation for microcontrollers and constrained systems

2.1.裸机 MicroPython 仓库结构

裸机第三方库 MicroPython 包含以下文件：

· extmod/ -- 跨多个端口共享的 CPython 或 MicroPython 特定模块。

· lib/ -- 包含第三方低级 C 库和 SDK。

· ports/ -- 特定于裸机 SDK 的接口

· py/ -- Micropython 源文件

· shared/ -- 包含开发的库、实用程序和帮助程序代码。

其中 ports 文件夹是用户文件夹，可根据用户自身情况进行添加或修改开发，其主要包括以下几个部分：

· genhdr/ -- 包含了 sdk 例程中可使用的模块和函数。

· user/ -- 用户自定义文件夹，可添加用户关注的模块和组件。

· _frozen_mpy.c：frozen 文件夹，有自己的 QSTR 值和池。

· mpconfigport.h：配置文件，相对应可使用的模块。

· mphalport.h：包括一些基本硬件相关函数的定义。

· mpy_main.c: micropython 启动跳转入口。

· mpy_uart.c: 与串口交互。

· mpy_uart.h: uart 相关定义。

2.2. 裸机 Micropython 特性支持

如前文所述，Micropython 是对 Python3 的一种高效实现，裁剪了大部分标准库，仅保留核心代码，Micropython 有以下关键特点：

- 支持多种架构，例如：x86, x86-64, ARM, ARM Thumb, Xtensa 等；
- 核心代码覆盖率为 98.4%，核心加扩展模块为 96.3%；
- 支持标记清除垃圾收集器，可用于堆内存管理；
- 支持 thread 操作；
- 直接针对机器代码而不是字节码虚拟机的本机发射器。

根据裸机环境以及 Micropython 特点，当前裸机代码仓库中支持以下关键特性：

Micropython 支持特性名	SDK 支持情况	功能说明
array	支持	数字数据的数组
binascii	支持	二进制/ASCII 转换
builtins	支持	内置函数和异常
collections	支持	集合和容器类型
errno	支持	系统错误代码
gc	支持	控制垃圾收集器
hashlib	支持	散列算法
heapq	支持	堆队列算法
io	支持	输入/输出流
json	支持	JSON 编码和解码
math	支持	数学函数
os	支持	基本的“操作系统”服务
random	支持	生成随机数
re	支持	简单的正则表达式

select	支持	等待一组流上的事件
struct	支持	打包和解包原始数据类型
sys	支持	系统特定的功能
cmath	暂不支持	复数的数学函数
socket	暂不支持	网络插座模块
ssl	暂不支持	SSL/TLS 模块
time	暂不支持	时间相关函数
uasyncio	暂不支持	异步 I/O 调度程序
zlib	暂不支持	zlib 解压
_thread	暂不支持	多线程支持

特定于 MicroPython 实现的功能：

Micropython 支持特性名	SDK 支持情况	功能说明
bluetooth	暂不支持	低级蓝牙
btree	暂不支持	简单的 BTree 数据库
cryptolib	暂不支持	密码
framebuff	暂不支持	帧缓冲区操作
machine	暂不支持	与硬件相关的功能
micropython	暂不支持	访问和控制 MicroPython 内部结构
neopixel	暂不支持	WS2812 / NeoPixel LED 的控制
network	暂不支持	网络配置
uctypes	暂不支持	以结构化的方式访问二进制数据

2.3. 例程使用说明及介绍

在 Phytium 系列 CPU，即 E2000 D/Q 和 FT2000/4 与 D2000 平台上，利用裸机仓库代码，Micropython 相关例程已实现在终端中进入 repl 命令行的功能，并利用文件系统访问板载 SD 卡，用户可在命令行界面输入基本 python 命令，而且可以通过 import 或 open 等文件访问接口访问各类格式文件。以下是例程相关使用说明：

在 repl 交互环境中，可进行基本 MicroPython 语法操作及调用 MicroPython 内置组件，实现 MicroPython 编程。

```
>>>
>>>
>>>
>>> help('modules')
__main__          errno             json              sd
_asyncio          frozentest        machine           select
array             gc                math              struct
binascii          hashlib           os                sys
builtins          heapq             random
collections       io                re
Plus any modules on the filesystem
>>>
>>>
>>> import math
>>>
>>> help(math)
object <module 'math'> is of type module
  __name__ -- math
  e -- 2.718282
  pi -- 3.141593
  sqrt -- <function>
  pow -- <function>
  exp -- <function>
  expm1 -- <function>
  log -- <function>
  log2 -- <function>
  log10 -- <function>
  cosh -- <function>
  sinh -- <function>
```

图 3-1 MicroPython 裸机环境运行初始图

当进入 repl 交互界面后，用户可使用基础 python 命令，并可使用 import 函数导入相关模块，可输入 help 函数查看模块相关功能及定义。

```
>>>
>>>
>>>
>>> help('modules')
__main__          errno             json              sd
_asyncio          frozentest        machine           select
array             gc                math              struct
binascii          hashlib           os                sys
builtins          heapq             random
collections       io                re
Plus any modules on the filesystem
>>>
>>>
>>> import math
>>>
>>> help(math)
object <module 'math'> is of type module
  __name__ -- math
  e -- 2.718282
  pi -- 3.141593
  sqrt -- <function>
  pow -- <function>
  exp -- <function>
  expm1 -- <function>
  log -- <function>
  log2 -- <function>
  log10 -- <function>
  cosh -- <function>
  sinh -- <function>
```

图 3-2　MicroPython 查看模块

此外，若想访问 SD 卡文件内容，可使用 open 等文件操作函数以及 import 函数访问并运行.py 文件，例如：

```
>>>
>>>
>>>
>>> import os
>>> os.listdir()
['sys', 'lib', 'cert', 'hello.py', 'System Volume Information', 'hello.txt']
>>>
>>>
>>> f = open('hello.txt')
>>>
>>> f.read()
'Hello this is micropython'
>>>
>>>
>>>
>>>
>>> import hello
Hello ,this is micropython !
>>>
```

图 3-3　MicroPython 文件操作

以上为打开并读取 SD 卡中 hello.txt 文件，以及运行 hello.py 文件。其余例如 write，close 等函数均可使用。

3.基于裸机 + MicroPython 开发指南

当使用 Micropython 进行开发时，使用者可能会遇到这样一些限制，比如官方没有实现自己想要的功能，或者你觉得这些实现不符合自己的工作需求。此刻，用户可能需要添加一些自定义的函数或模块以实现自身特定需求。下面将介绍如何在本平台中开发自定义组件和自定义函数。

3.1 如何添加用户函数

自定义函数实现默认是添加在已有的模块函数库中，例如在已添加的 builtins 模块中添加一个类 add 函数，一共需要以下几步：

- 首先需要在想注册的模块中添加自定义函数实现，例如在模块文件 modbuiltins.c 中添加：

```
STATIC mp_obj_t my_function(mp_obj_t arg_1_obj, mp_obj_t arg_2_obj)
{
    mp_int_t arg_1 = mp_obj_get_int(arg_1_obj);
    mp_int_t arg_2 = mp_obj_get_int(arg_2_obj);
    mp_int_t ret_val;
    ret_val = arg_1 + arg_2;
    return mp_obj_new_int(ret_val);
}
MP_DEFINE_CONST_FUN_OBJ_2(my_function_obj, my_function);
```

此函数实现类 add 函数，返回输入值相加后的值。

- 将函数注册到模块函数列表中：

```
STATIC const mp_rom_map_elem_t mp_module_builtins_globals_table[] =
{ { MP_ROM_QSTR(MP_QSTR___name__),MP_ROM_QSTR(MP_QSTR_builtins) },
{ MP_ROM_QSTR(MP_QSTR_my_function),MP_ROM_PTR(&my_function_obj) },
  // built-in core functions
{ MP_ROM_QSTR(MP_QSTR___build_class__),MP_ROM_PTR(&mp_builtin___build
_class___obj) },
{ MP_ROM_QSTR(MP_QSTR___import__),MP_ROM_PTR(&mp_builtin___import__
_obj) },
{ MP_ROM_QSTR(MP_QSTR___repl_print__),MP_ROM_PTR(&mp_builtin___repl_p
rint___obj) },
```

在注册后，import 相关 module 可查看到模块下可使用的函数列表。

- 生成 QSTR 代码值并追加到<port/genhdr/qstrdefs.generated.h> 文件后

```
QDEF0(MP_QSTR_my_function, 16748, 11, "my_function")
```

- 烧录并 import 查看模块下函数实现是否可用

```
>>>
>>>
>>>
>>>
>>>
>>>
>>> import builtins
>>> builtins.my_function(121,121)
242
>>>
```

图 3-4 自定义函数

注：具体如何生成自定义 QSTR 值，可将以下代码块添加至在线 C 语言工具(C 在线工具 | 菜鸟工具)中运行生成，此处由 micropthon 源码提取生成 C 函数。

```
#include <stdio.h>
#include <string.h>
unsigned int qstr_compute_hash(const char *data, unsigned int len);
int main()
{
    #define STR_SIGNLE_LEN (50)
    const unsigned char str_group[][STR_SIGNLE_LEN] =
    {
        "my_function"
    };
    for(int i = 0; i < sizeof(str_group)/STR_SIGNLE_LEN; i++)
    {
        const unsigned char *str_p = str_group[i];
        const unsigned int str_len = strlen(str_p);
        if(str_len > 0)
        {
            unsigned int hash = qstr_compute_hash(str_p, str_len);
            unsigned char str_out[100] = {0};
            sprintf(str_out, "QDEF0(MP_QSTR_%s, %d, %d, \"%s\")", str_p, hash,
str_len, str_p);
            printf("%s\r\n", str_out);
        }
        else
        {
```

```
                break;
            }
        }
        return 0;
}
#define MICROPY_QSTR_BYTES_IN_HASH (2)
#define Q_HASH_MASK ((1 << (8 * MICROPY_QSTR_BYTES_IN_HASH)) - 1)
unsigned int qstr_compute_hash(const char *data, unsigned int len) {
    // djb2 algorithm; see http://www.cse.yorku.ca/~oz/hash.html
    unsigned int hash = 5381;
    for (const char *top = data + len; data < top; data++) {
        hash = ((hash << 5) + hash) ^ (*data); // hash * 33 ^ data
    }
    hash &= Q_HASH_MASK;
    // Make sure that valid hash is never zero, zero means "hash not computed"
    if (hash == 0) {
        hash++;
    }
    return hash;
}
```

3.2 如何添加用户模块

自定义模块是为了方便用户添加 Micropython 库中不存在的模块，或者用户想添加新的组件进行调试外设。下面以添加一个 led 模块为例：

- 首先在 ports/user/文件夹下建立一个 my_module.c 文件并确保其在编译时被包含，其文件内容至少包括，用户可扩展模块函数，可参照节 6.1 或仓库其他模块实现，需注册在 led_module_globals_table[]列表中：

```
STATIC const mp_rom_map_elem_t led_module_globals_table[] = {
{MP_ROM_QSTR(MP_QSTR___name__),MP_ROM_QSTR(MP_QSTR_led) },
};
STATIC MP_DEFINE_CONST_DICT(led_module_globals, led_module_globals_table);
const mp_obj_module_t led_module = {
    .base = { &mp_type_module },
    .globals = (mp_obj_dict_t *)&led_module_globals,
};
```

- 在 ports/genhdr/moduledefs.collected 中添加

```
MP_REGISTER_MODULE(MP_QSTR_led, led_module);
```

- 在 ports/genhdr/moduledefs.h 中添加

```
extern const struct _mp_obj_module_t mp_module_led;
#undef MODULE_DEF_LED
#define MODULE_DEF_LED { MP_ROM_QSTR(MP_QSTR_led),
MP_ROM_PTR(&mp_module_led) },

#define MICROPY_REGISTERED_MODULES \
    MODULE_DEF_BUILTINS \
    MODULE_DEF_GC \
    MODULE_DEF_MATH \
    MODULE_DEF_LED \
```

- 在 ports/genhdr/qstrdefs.generated.h 中添加

```
QDEF0(MP_QSTR_led, 16488, 3, "led")
```

此字符串索引生成方式与函数生成方式一致，只需更改相应名字即可。

- 在 repl 界面中查看 import 导入是否正常

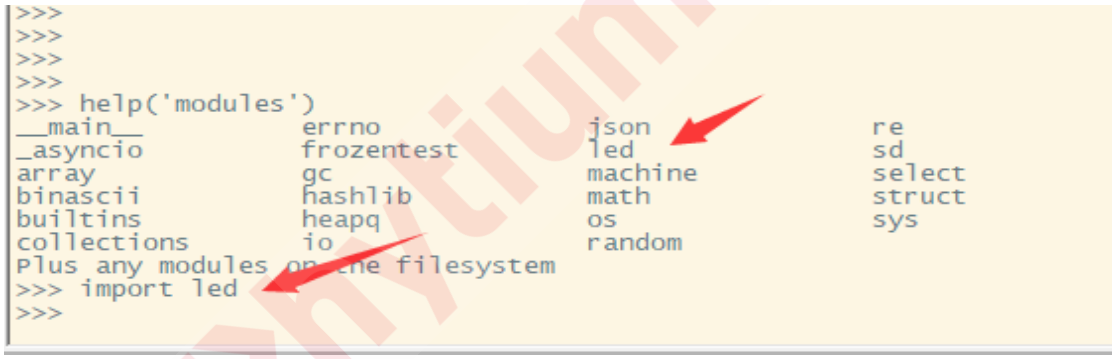

图 3-5 自定义函数

当导入模块无错误异常抛出，用户即可使用本模块进行相应开发。

4.基于裸机 + MicroPython 开发前瞻

MicroPython 为嵌入式开发提供了一个强大而灵活的工具，使开发人员能够使用 Python 语言构建嵌入式应用程序。

当前 MicroPython 越来越得到更多嵌入式平台厂商的关注，诸如 esp32,stm32,raspberry pi 等均对 MicroPython 仓库作了支持，基于 MicroPython 开发的应用也层出不穷，特别是在一些智能工业自动化解

决方案上效果明显。如基于 MicroPython + ESP32 的物联网监控系统，人脸跟踪机器人，Haar 特征分类器目标跟踪系统等；基于 MicroPython + raspberry pi 的 MQTT 服务以及基于 MicroPython + LVGL 的图形开发等等。

基于 Phytium 系列 CPU 以及裸机代码仓库和 FreeRTOS 软件开发工具包，用户可利用 MicroPython 强大特性进行应用开发。

A 参考文献

B 更新记录

发布日期	说明
2024-01-23	初稿

Phytium飞腾